🚀 Complete Backend & Architecture Guide: Subscription Tracker API

Welcome to your comprehensive backend learning guide! This document explains **every command you ran**, **every file in your project**, **how each line of code works under the hood**, and **why the architecture is built this way**.

📑 Table of Contents

1. Initial Setup Commands: What & Why
2. Module Systems: CommonJS vs. ES Modules (`"type": "module"`)
3. Deciphering `package.json` & Scripts
4. Environment Management: `.env` and `config/env.js`
5. Database Connectivity: `database/db.js`
6. Data Modeling with Mongoose
    - User Model (`models/user.model.js`)
    - Subscription Model (`models/subscription.model.js`)
7. The Application Core: `app.js`
8. Routing Layer: `routes/`
9. Error Handling Middleware: `middlewares/error.middleware.js`
10. The Controller Layer & Authentication Architecture (`controllers/auth.controller.js`)
11. Express Middleware Pipeline Order: Why Placement Matters
12. The Complete Request-Response Lifecycle
13. Next Action Items to Complete Your API

1. Initial Setup Commands: What & Why

🔹 Command: `npx express-generator --no-view --git ./`

- `npx` **(Node Package eXecute)**: Executes an npm package binary directly without permanently installing it globally on your computer.

- **`express-generator`** : An official scaffolding CLI utility created by the Express team that automates creating the directory structure, starter code, and default configurations.
- **`--no-view`** : By default, `express-generator` configures server-side HTML rendering engines (like Pug/Jade or EJS). Because you are creating a **REST API** (which transmits pure JSON to web clients, mobile apps, or Postman), you do not need server-rendered HTML views. This flag strips out view templates.
- **`--git`** : Automatically creates a `.gitignore` tailored for Node.js projects, ensuring bulky folders like `node_modules` and sensitive files like `.env` are never pushed to public Git repositories.
- **`./`** : Tells the generator to scaffold the files directly inside the **current directory**, rather than creating a sub-folder.

🔹 Command: `npm install --save-dev nodemon`

- **Why `nodemon` ?**: By default in Node.js, when you run `node app.js` , the process reads files into memory and stays running. If you edit code, your changes **will not apply** until you manually terminate the server (`Ctrl + C`) and restart it. `nodemon` monitors your project directory for file modifications and automatically reboots the server the instant you save (`Ctrl + S`).
- **Why `--save-dev` ?**: Installs the package under `devDependencies` in `package.json` . In production (e.g. AWS, Render, Vercel), the live server should run via standard `node app.js` without file watchers. Tools needed exclusively during development belong in `devDependencies` .

🔹 Command: `npx eslint --init` & `eslint.config.js`

- **What is ESLint?**: A static code analysis tool that checks your code for syntax errors, undeclared variables, formatting issues, and bad practices before runtime.
- **Flat Config (`eslint.config.js`)**: Modern ESLint (v9+) uses the flat config format. It defines which files to inspect (`files: ["**/*.{js,mjs,cjs}"]`) and tells ESLint to recognize standard Node.js global variables (`process` , `console` , `Buffer`) via `globals.node` .

🔹 Command: `npm install dotenv mongoose`

- **`dotenv`** : A lightweight library that loads environment variables from a `.env` file into Node.js's native `process.env` global object.

- `mongoose` : An Object Data Modeling (ODM) library for MongoDB. While MongoDB natively stores flexible, schema-less BSON documents, `mongoose` gives structure to your data through schemas, strict type definitions, built-in validation rules, query helpers, and middleware hooks.

2. Module Systems: CommonJS vs. ES Modules

When you add `"type": "module"` in `package.json` , you tell Node.js to use modern ECMAScript (ES) Modules instead of legacy CommonJS.

Comparison Area	**CommonJS (Default / Legacy)**	**ES Modules (`"type": "module"`)**
Import Syntax	`const express = require('express');`	`import express from 'express';`
Export Syntax	`module.exports = app;`	`export default app;` or `export { app };`
Asynchronous Loading	Synchronous execution	Asynchronous, statically analyzable
File Extensions	Optional (`require('./db')`)	**Mandatory** for local files (`import db from './db.js'`)
Top-Level `await`	Not supported in standard CJS files	Supported out of the box

> **Important**
>
> **ESM Gotcha in Node.js**:
> When importing external npm packages (e.g., `import express from 'express'`), you don't use extensions.
> But when importing your own local files, **you must include the `.js` extension**:

```
// ❌ Causes runtime error in Node.js ESM:
import db_connection from './database/db';

// ✅ Correct:
import db_connection from './database/db.js';
```

3. Deciphering `package.json` & Scripts

Here is your current `package.json`:

```
{
  "name": "subscription-tracker",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "cookie-parser": "~1.4.4",
    "debug": "~2.6.9",
    "dotenv": "^17.4.2",
    "express": "~4.16.1",
    "mongodb": "^7.6.0",
    "mongoose": "^9.9.5",
    "morgan": "~1.9.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "eslint": "^10.10.0",
    "globals": "^17.12.0",
    "nodemon": "^3.1.14"
  }
}
```

The Scripts:

- **`npm start`** : The production command (`node app.js`). Node directly executes the application once.
- **`npm run dev`** : The developer command (`nodemon app.js`). Nodemon spawns a child process and restarts it whenever you edit files.

4. Environment Management: `.env` and `config/env.js`

Your `config/env.js` Code:

```
import { config } from "dotenv";

config({path: `.env.${process.env.NODE_ENV || 'development'}.local`});

export const { PORT, NODE_ENV, MONGODB_URI } = process.env;
```

Deep Dive: Why is this written this way?

1. **Dynamic Environment Configuration**:

   ```
   `.env.${process.env.NODE_ENV || 'development'}.local`
   ```

   - If `process.env.NODE_ENV` is not set, it defaults to `'development'` , resolving to `.env.development.local` .
   - When deployed to production, `NODE_ENV=production` automatically switches loading to `.env.production.local` .
   - This prevents accidental mixups between your local test database and your live production database.
2. **Centralized Variable Export**:

   ```
   export const { PORT, NODE_ENV, MONGODB_URI } = process.env;
   ```

   Instead of sprinkling `process.env.PORT` across 10 different files, you import clean, destructured constants:

   ```
   import { PORT } from './config/env.js';
   ```

If a variable name changes in the future, you only have to update it in one place!

5. Database Connectivity: `database/db.js`

Your `database/db.js` Code:

```
import mongoose from "mongoose";
import { MONGODB_URI } from "../config/env.js";

if (!MONGODB_URI) {
    throw new Error('Please provide MONGODB_URI');
}

const db_connection = async () => {
    try {
        await mongoose.connect(MONGODB_URI);
        console.log('Database connected');
    } catch (error) {
        console.log(error);
        process.exit(1);
    }
}

export default db_connection;
```

Deep Dive:

- **Pre-flight Guard (`if (!MONGODB_URI)`)**: Prevents silent connection failures. If someone forgot to configure the `.env` file, the server fails immediately with an informative message.
- **Async/Await**: `mongoose.connect()` returns a Promise because establishing a network socket connection to MongoDB Atlas takes time. `await` halts execution of the function until the handshake completes.
- **Why `process.exit(1)` ?**: In Node.js:
  - Exit code `0` indicates normal, clean termination.
  - Exit code `1` (or any non-zero code) indicates an uncaught error or fatal failure.
    If the database is unreachable, a database-driven API cannot function. It is much safer to

terminate the server process immediately so process managers (like Docker, Kubernetes, or PM2) can alert the developer or restart the container.

6. Data Modeling with Mongoose

User Model (`models/user.model.js`)

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Name is required'],
        maxLength: 50,
        minLength: 2,
    },
    email: {
        type: String,
        required: [true, 'Email is required'],
        unique: true,
        lowercase: true,
    },
    password: {
        type: String,
        required: [true, 'Password is required'],
        minlength: 6,
    }
}, { timestamps: true });

const User = mongoose.model('User', userSchema);
export default User;
```

Key Mechanics:

1. **Validation Arrays (`[true, 'Custom message']`)**: Providing a custom string allows Mongoose to return human-friendly error messages when a field is missing.
2. **`unique: true` on `email`** : Under the hood, Mongoose commands MongoDB to construct a **Unique B-Tree Index** on the `email` field. If two users try to register with the same email, MongoDB rejects the second insert with a duplicate key error (`E11000`).

3. **`lowercase: true`** : Automatically converts emails like `User@Example.COM` to `user@example.com` before saving, eliminating casing bugs in lookups.
4. **`{ timestamps: true }`** : Instructs Mongoose to automatically append two `Date` fields to every document:
    - `createdAt` : When the document was inserted.
    - `updatedAt` : Updated whenever the document is modified.

Subscription Model (`models/subscription.model.js`)

This model contains advanced Mongoose features:

```
import mongoose from "mongoose";

const subscriptionSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Subscription name is required'],
        trim: true,
        minLength: 2,
        maxLength: 30
    },
    price: {
        type: Number,
        required: [true, 'Amount is required'],
        min: [0, 'Price must be greater than 0'],
        max: [1000, 'Price must be less than 1000']
    },
    currency: {
        type: String,
        enum: ['USD', 'INR', 'YEN'],
        default: 'USD'
    },
    frequency: {
        type: String,
        enum: ['daily', 'weekly', 'monthly', 'yearly']
    },
    category: {
        type: String,
        enum: ['sports', 'news', 'entertainment', 'music', 'education', 'tool', 'other'],
        required: true
    },
    paymentMethod: {
        type: String,
        required: true,
        trim: true
    },
    status: {
        type: String,
        enum: ['PENDING', 'ACTIVE', 'INACTIVE'],
        default: 'ACTIVE'
    },
    startDate: {
        type: Date,
        required: true,
```

```
        validate: {
            validator: function (value) {
                return value <= new Date();
            },
            message: 'Start Date cannot be in the future'
        }
    },
    renewalDate: {
        type: Date,
        validate: {
            validator: function (value) {
                return value >= this.startDate;
            },
            message: 'Renewal Date cannot be in the past'
        }
    },
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: [true, 'User is required'],
        index: true
    }
}, { timestamps: true });
```

Advanced Features:

1. **`enum` (Enumerations)**: Restricts valid values to a whitelist. If someone sends `"frequency": "decade"`, Mongoose blocks the write.
2. **`trim: true`**: Automatically removes accidental leading or trailing whitespace (`"  Netflix "` becomes `"Netflix"`).
3. **Custom Validators (`validate`)**: Functions that return a boolean. Notice how `renewalDate` accesses `this.startDate` to ensure the renewal date is chronologically after the start date.
4. **Relational Referencing (`ObjectId` + `ref: 'User'`)**:
    - Stores the 24-character hexadecimal `_id` of the User who owns the subscription.
    - `index: true` creates an index on `userId`, making queries like `Subscription.find({ userId: req.user._id })` lightning fast even with millions of records.
    - Allows using Mongoose's `.populate('userId')` to automatically fetch user details in queries.

The Auto-Calculation Hook (`pre('save')`):

```
subscriptionSchema.pre('save', function (next) {
    if (!this.renewalDate) {
        const renewalPeriods = {
            daily: 1,
            weekly: 7,
            monthly: 30,
            yearly: 365,
        };

        this.renewalDate = new Date(this.startDate);
        this.renewalDate.setDate(this.renewalDate.getDate() + renewalPeriods[this.frequency]);
    }

    if (this.renewalDate < new Date()) {
        this.status = 'EXPIRED';
    }

    next();
});
```

- **What is `pre('save')` ?**: A document middleware hook that triggers automatically right before a document is written into the MongoDB database.
- **Auto-renewal Date Calculation**: If the user didn't specify a `renewalDate` , the code calculates it by reading `this.startDate` and adding the interval days according to `this.frequency` .
- **Auto-expiry**: If the renewal date is already in the past, it flips `this.status` to `'EXPIRED'` .
- **Why regular function instead of arrow function `() => {}` ?**:
  In arrow functions, `this` is lexically bound to the parent scope. In a Mongoose hook, you **must use `function () {}`** so that `this` points to the **Mongoose document being saved**!
- **`next()`** : Passes control to the next hook or writes the document to MongoDB.

7. The Application Core: `app.js`

```
import express from 'express';
import { PORT } from './config/env.js';
import auth_router from './routes/auth.route.js';
import db_connection from './database/db.js';

const app = express();

// Body Parser Middleware
app.use(express.json());

// Routes
app.use('/api/v1/auth', auth_router);

// Health Check
app.get('/', (req, res) => {
    res.send('Working Mode On');
});

// Server Initialization
app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
    db_connection();
});
```

Deep Dive:

1. **`app.use(express.json())`** : When a client sends a `POST` request with JSON data in the body, HTTP sends it over the network as a raw stream of bytes. This middleware intercepts the stream, buffers it, parses the JSON string, and attaches it as a plain JavaScript object to `req.body` .
2. **API Versioning (`/api/v1/auth`)**: Prefixing routes with `/api/v1/` allows you to introduce breaking changes in the future (e.g. `/api/v2/`) without breaking existing mobile apps or frontend clients that rely on v1.
3. **Connecting DB inside `app.listen()`** : By placing `db_connection()` inside the `app.listen()` callback, the database connects as soon as the port opens.

8. Routing Layer: `routes/`

Express provides `Router()` to split endpoints across separate modular files:

```
routes/
├── auth.route.js          # Authentication (signup, login, logout)
├── user.route.js          # User profile operations
└── subscription.route.js  # Subscription management
```

RESTful URL Conventions:

In a production REST API, URLs identify **resources** (nouns), and HTTP verbs identify **actions**:

Action	Recommended RESTful Route	HTTP Verb	Purpose
Get All	`/api/v1/subscriptions`	`GET`	Retrieve subscriptions list
Get One	`/api/v1/subscriptions/:id`	`GET`	Retrieve single subscription
Create	`/api/v1/subscriptions`	`POST`	Create a new subscription
Update	`/api/v1/subscriptions/:id`	`PUT` / `PATCH`	Update an existing subscription
Delete	`/api/v1/subscriptions/:id`	`DELETE`	Remove a subscription

> **Tip**
>
> In your current `subscription.route.js`, you have `/new`, `/:id/edit`, and `/:id/delete` using `GET`.
> That pattern is used in traditional server-rendered HTML apps (where browser forms only support `GET` and `POST`).
> For a **REST API**, use proper HTTP methods:

- `POST /api/v1/subscriptions` instead of `GET /new`
- `PUT /api/v1/subscriptions/:id` instead of `GET /:id/edit`
- `DELETE /api/v1/subscriptions/:id` instead of `GET /:id/delete`

9. Error Handling Middleware:

`middlewares/error.middleware.js`

In Express, an error-handling middleware **must accept 4 arguments**:

```
(err, req, res, next) => { ... }
```

If you omit any of the 4 arguments (even `next`), Express will treat it as regular middleware instead of an error handler.

The Problem Solved in `Undestanding_things.md`:

```
const errorMiddleware = (err, req, res, next) => {
    try {
        let error = { ...err };
        error.message = err.message;

        console.log(err);

        // 1. Mongoose Bad ObjectId (CastError)
        if (err.name === 'CastError') {
            const message = "Resource not found";
            error = new Error(message);
            error.statusCode = 404;
        }

        // 2. Mongoose Duplicate Key (Code 11000)
        if (err.code === 11000) {
            const message = "Duplicate field value entered";
            error = new Error(message);
            error.statusCode = 400;
        }

        // 3. Mongoose Validation Error
        if (err.name === 'ValidationError') {
            const message = Object.values(err.errors).map(val => val.message);
            error = new Error(message.join(', '));
            error.statusCode = 400;
        }

        res.status(error.statusCode || 500).json({
            success: false,
            error: error.message || 'Server Error'
        });
    } catch (error) {
        next(error);
    }
};

export default errorMiddleware;
```

Key Questions & Answers:

1. Why `let error = { ...err }` and `error.message = err.message` ?

- In JavaScript, standard `Error` objects store `name` , `message` , and `stack` on the prototype chain, and they are defined as **non-enumerable**.
- The object spread operator `{ ...err }` copies **only enumerable own properties** (like Mongoose's `err.code` , `err.errors`).
- It **fails to copy `message` and `name`** . That is why `error.message = err.message` must be assigned explicitly.

2. Why `console.log(err)` instead of `console.log(error)` ?

- `err` is the original native `Error` instance with stack trace, prototype formatting, and error symbols. Node's console inspector formats it cleanly with line numbers and file paths.
- `error` is just a plain `{}` object; logging it would hide the stack trace.

3. Why `if (err.name === 'CastError')` instead of `error.name` ?

- Because `name` was not copied during `{ ...err }` , `error.name` is `undefined` . Checking `err.name` inspects the real error prototype.

10. The Controller Layer & Authentication Architecture (`controllers/auth.controller.js`)

In an MVC architecture, the **Controller** acts as the business orchestrator. It sits between incoming HTTP routes and database models, validating input, orchestrating business rules, managing database transactions, and returning sanitized responses.

Detailed Code Breakdown of `signup` :

```
import mongoose from "mongoose";
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import User from "../models/user.model.js";
import { JWT_EXPIRES_IN, JWT_SECRET } from "../config/env.js";

export const signup = async (req, res, next) => {
    const session = await mongoose.startSession();
    session.startTransaction();

    try {
        const { name, email, password } = req.body;

        // 1. Check if user already exists
        const existing_user = await User.findOne({ email });

        if (existing_user) {
            const error = new Error("User already exists.");
            error.statusCode = 409;
            throw error;
        }

        // 2. Hash Password
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(password, salt);

        // 3. Create User inside transaction
        const newUsers = await User.create([{
            name: name,
            email: email,
            password: hashedPassword,
        }], { session });

        // 4. Generate JWT Token
        const token = jwt.sign({ userId: newUsers[0]._id }, JWT_SECRET, { expiresIn: JWT_EXPIRES

        // 5. Commit transaction & end session
        await session.commitTransaction();
        session.endSession();

        // 6. Send Response
```

```
        res.status(201).json({
            success: true,
            message: 'User created',
            data: {
                token: token,
                user: newUsers[0]
            }
        });

    } catch (error) {
        // Rollback all database operations if anything fails
        await session.abortTransaction();
        session.endSession();
        next(error);
    }
}
```

🔹 Deep Dive: Key Concepts in `auth.controller.js`

1. Multi-Document ACID Transactions (`session`)

- **What are Transactions?**: In real-world backends, creating a user might also create an initial billing profile or trigger subscription records. If any step fails, leaving a partially created user document corrupts your database.
- **`startSession()` & `startTransaction()`** : Starts an isolated MongoDB transaction sandbox.
- **`await session.commitTransaction()`** : Flushes all staged database writes permanently to MongoDB Atlas.
- **`await session.abortTransaction()`** : If any error is thrown in the `try` block, execution jumps directly to `catch` , rolling back any writes performed during this session.
- **Why documents must be in an array `User.create([{ ... }], { session })`** :
  When passing an options object like `{ session }` to `Model.create()` , Mongoose expects the documents to be wrapped in an array `[{ ... }]` . Otherwise, Mongoose mistakes `{ session }` as a second user document to save.

2. Password Security: Salting & Hashing (`bcryptjs`)

- **Plaintext is strictly forbidden**: Never store raw passwords or use weak hashes like MD5/SHA-256.
- **Why Salt? (`bcrypt.genSalt(10)`)**: A salt is random cryptographic data appended to the password before hashing. Even if two users have the identical password (`"Password123"`), their salted hashes will be completely different, defeating precomputed rainbow tables.

- **Cost Factor (10 rounds)**: $2^{10} = 1024$ calculation iterations create an intentional micro-delay, making brute-force dictionary attacks practically impossible while taking milliseconds for real users.

3. Stateless Token Authentication (`jsonwebtoken`)

- **Why JWT over Server Sessions?**: Traditional sessions require server memory or shared databases (like Redis). JWTs are cryptographically signed tokens held by the client, allowing your backend to scale horizontally without maintaining session state.
- **Payload**: `{ userId: newUsers[0]._id }` . Only store non-sensitive identifiers in JWT payloads—never store passwords, credit cards, or internal secret keys.
- `JWT_SECRET` : The secret key known only to the backend server. The signature prevents clients or attackers from modifying the token payload.
- `JWT_EXPIRES_IN` : Enforces token expiration (e.g., `'1d'`), limiting the danger if a token is ever stolen.

4. Security Tip: Sanitizing User Objects

- In `res.status(201).json(...)` , `newUsers[0]` still includes the `password` hash field. In a production environment, sanitize the object before returning it:

```
const userResponse = newUsers[0].toObject();
delete userResponse.password;
```

5. The `await` vs. Missing `await` Trap (`User.findOne`)

- **What happens WITHOUT `await`** :

```
// ❌ Dangerous bug:
const existing_user = User.findOne({ email });
```

`User.findOne()` returns an unexecuted **Mongoose Query object** (an internal JavaScript object instance containing query configs, models, and filters). In JavaScript, all objects are truthy (`Boolean({}) === true`).
Consequently:

```
if (existing_user) { ... } // ALWAYS EVALUATES TO TRUE!
```

Even if the user does NOT exist in MongoDB, the condition triggers and throws `"User already exists"` , completely breaking user registration!

- **What happens WITH `await`** :

```
// ✅ Correct:
const existing_user = await User.findOne({ email });
```

`await` tells JavaScript to pause the async function until MongoDB executes the query over the network and returns the result:

- If user exists: MongoDB returns the `User` document (truthy) ➡️ enters `if` and throws 409 Conflict.
- If user does not exist: MongoDB returns `null` (falsy) ➡️ skips `if` and allows sign-up to proceed.

6. Why Documents Must Be in an Array: `User.create([ ... ], { session })`

- When passing transaction options like `{ session }` to Mongoose's `Model.create()`, Mongoose expects documents to be passed inside an array:

```
// ✅ Correct:
const newUsers = await User.create([{
    name: name,
    email: email,
    password: hashedPassword,
}], { session });
```

- **What happens without the array `[ ]`:**
  If you pass `User.create({ name, ... }, { session })`, Mongoose treats `{ session }` as a **second user document** to insert rather than transaction options!
  Furthermore, `newUsers[0]` will be `undefined`, causing `jwt.sign({ userId: newUsers[0]._id }, ...)` to throw a fatal runtime error (`Cannot read properties of undefined (reading '_id')`).

7. `req` vs. `res` in Controller Responses

- `req` **(Incoming HTTP Request)**: Represents incoming data from the client (headers, URL parameters, query string, and `req.body`). It does **not** have response helper methods. Calling `req.status(201)` crashes the server with `TypeError: req.status is not a function`.
- `res` **(Outgoing HTTP Response)**: Created by Express to format and transmit data back over the HTTP connection using methods like `res.status(code).json({ ... })` or `res.send()`.

8. Connection Pooling & Why `session.endSession()` is Mandatory

- Sessions allocate dedicated state, memory, and socket connections inside MongoDB's connection pool.

- If you do not call `session.endSession()`, especially when errors occur, the connection pool leaks. Eventually, the database runs out of available connections and rejects all incoming traffic.
- That is why `session.endSession()` is executed inside both the completion path and the `catch` (or `finally`) block.

11. Express Middleware Pipeline Order: Why Placement Matters

In Express, **the order in which middleware and routes are defined is law**. Incoming requests flow top-to-bottom through your middleware pipeline.

```
Incoming Request
      |
      ▼
1. express.json()            <-- Parses JSON body
      ▼
2. cookieParser()            <-- Parses cookies
      ▼
3. Routes (/api/v1/auth, ...) <-- Match and execute route controllers
      ▼
4. errorMiddleware           <-- Catches and formats any errors passed via next(err)
```

⚠️ Common Bug in `app.js`:

Notice if a route is mounted after the error middleware:

```
// ❌ INCORRECT ORDER:
app.use(errorMiddleware);

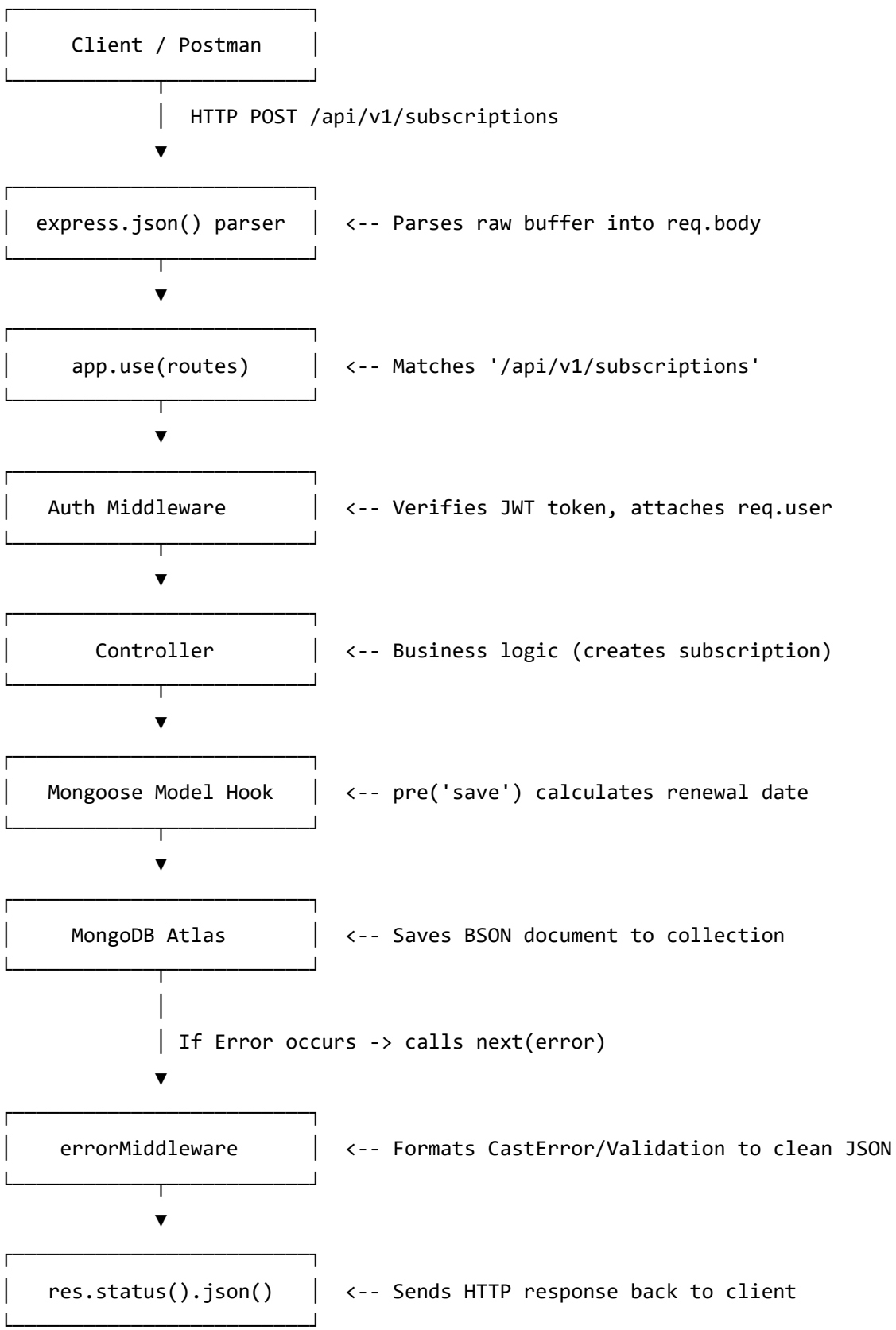
app.get('/', (req, res) => {
    res.send('Working Mode On');
});
```

- If `app.get('/')` is declared **after** `app.use(errorMiddleware)`, any error thrown inside that route cannot be handled by `errorMiddleware` because the request has already passed it!
- **Best Practice Order**:
    i. Top: Parsers (`express.json()`, `cookieParser()`) and loggers (`morgan`).

ii. Middle: All API routes (`/api/v1/auth` , `/api/v1/users` , `/api/v1/subscriptions` , and root `/`).
iii. Bottom: Centralized `errorMiddleware` (must always be the final `app.use()` call).

12. The Complete Request-Response Lifecycle

Here is the exact journey a request takes in your application:


Client / Postman
HTTP POST /api/v1/subscriptions
express.json() parser
<-- Parses raw buffer into req.body
app.use(routes)
<-- Matches '/api/v1/subscriptions'
Auth Middleware
<-- Verifies JWT token, attaches req.user
Controller
<-- Business logic (creates subscription)
Mongoose Model Hook
<-- pre('save') calculates renewal date
MongoDB Atlas
<-- Saves BSON document to collection
If Error occurs -> calls next(error)
errorMiddleware
<-- Formats CastError/Validation to clean JSON
res.status().json()
<-- Sends HTTP response back to client

13. Next Action Items to Complete Your API

1. ✅ **Mount all routes in `app.js`** *(Completed)*:
   `auth.route.js`, `user.route.js`, and `subscription.route.js` are mounted under `/api/v1/`.
2. ✅ **Mount `errorMiddleware` in `app.js`** *(Completed)*:
   Global error handler catches validation, duplicate key, and bad ObjectId errors. *(Note: Ensure `app.get('/')` is placed above `app.use(errorMiddleware)`).*
3. 🟡 **Complete Authentication Controllers (`controllers/auth.controller.js`)**:
   - ✅ `signup` implemented with transactions, bcrypt password hashing, and JWT creation.
   - ⬜ `signin`: Verify credentials with `bcrypt.compare` and return a JWT.
   - ⬜ `signout`: Clear client cookies or tokens.
4. ⬜ **Add Protected Route Middleware (`middlewares/auth.middleware.js`)**:
   Intercept requests, extract the JWT from the `Authorization: Bearer <token>` header or cookies, verify it with `jwt.verify`, and attach the logged-in user to `req.user`.
5. ⬜ **Implement Subscription Controllers (`controllers/subscription.controller.js`)**:
   CRUD operations for user subscriptions (create subscription tied to `req.user._id`, view subscriptions, update status, cancel).